SECTION 1 — LINUX BASICS & OS CONCEPTS

1. What is the role of the kernel in an operating system?

The kernel is the core of the OS — it mediates between hardware and user-space programs. Responsibilities: process management (scheduling, context switch), memory management (virtual memory, page tables, allocation), device driver framework (abstract hardware), inter-process communication (signals, pipes, sockets), filesystem management (VFS), and enforcing protection/security. The kernel provides system calls as the API that user programs use to request services.

2. What is the difference between a monolithic kernel and a microkernel?

- *Monolithic kernel*: Most OS services (drivers, filesystems, network stack) run in kernel space as a single large binary (Linux). Pros: performance (no IPC overhead), simpler in some ways. Cons: larger TCB, bugs in drivers can crash kernel.
- *Microkernel*: Keeps only minimal services (scheduling, IPC, memory mgmt) in kernel; other services (drivers, filesystems) run in user space and talk via IPC (e.g., QNX, MINIX). Pros: better isolation and modularity. Cons: more IPC overhead and design complexity.

3. Explain the process scheduling in Linux.

Modern Linux uses the Completely Fair Scheduler (CFS) for normal tasks and specialized schedulers (SCHED_FIFO, SCHED_RR) for realtime. CFS models fairness using a red-black tree keyed by *vruntime* (virtual runtime) so each runnable task gets a proportional share. Scheduler responsibilities: pick next task, manage priorities/nice values, enforce scheduling policies, support load balancing across CPUs (SMP), and handle wakeups. CFS aims to minimize latency and maximize fairness.

4. What is context switching?

Context switch is the act of saving the CPU state (registers, program counter, stack pointer, kernel thread state) of the currently running process/thread and loading the saved state of another process/thread. It allows multitasking. It involves TLB considerations, possible page-table switch (address space), and kernel bookkeeping. The cost includes CPU cycles and cache/TLB penalties.

5. How is memory management handled in Linux?

Linux provides virtual memory via per-process page tables and an MMU. Kernel

components: page allocator (zone allocator), slab/SLUB caches for objects, vmalloc for non-contiguous kernel allocations, swapbacking, page reclamation (kswapd), and memory cgroup enforcement. For user-space, Linux maps files and anonymous memory into address space, tracks protection bits, and uses demand paging. Memory protection and isolation are enforced by the kernel.

SECTION 2 — DEVICE DRIVERS

6. What is a character device driver?

A character device provides byte-stream access — users read/write as a stream (like /dev/tty, /dev/ttyS0). Character drivers implement file_operations callbacks (open, read, write, ioctl, mmap, release). They are appropriate for devices that do not require block I/O semantics (random-access fixed-size blocks).

7. What is a block device driver?

Block devices provide block-oriented access (fixed-size blocks) and support request queues, buffering and caching (e.g., /dev/sda). Block drivers implement the block device interface, provide a request handling routine (blk_mq modern path), and must manage I/O scheduling, partitions, and safe concurrency to enable filesystems.

8. Explain the steps to write a Linux character driver.

High-level steps:

1. Create module skeleton with module_init / module_exit.
2. Allocate or request a major/minor (register_chrdev_region or alloc_chrdev_region).
3. Initialize cdev (cdev_init, cdev_add).
4. Implement struct file_operations (open/read/write/unlocked_ioctl/mmap/release). Use correct locking and copy_to_user/copy_from_user.
5. Create class/device for udev (class_create, device_create) to make /dev node.
6. Handle error paths and module_param if config params needed.
7. On unload, reverse: device_destroy, class_destroy, cdev_del, unregister_chrdev_region. Test on target hardware.

9. What is the purpose of file_operations structure?

struct file_operations is the kernel's table of callbacks for a device or inode. It maps user

syscall operations (open, read, write, ioctl, mmap, poll, etc.) to driver functions. When a user program calls read(fd, ...), VFS dispatches to the .read function pointer in this structure.

10. How is the driver's entry and exit handled?

Use module_init(my_init) and module_exit(my_exit) to register entry/exit functions. Inside init: allocate resources, register device numbers, register cdev, create device nodes. In exit: free resources, unregister device numbers, remove cdev, destroy device nodes. Always ensure idempotency and free on error paths.

11. Difference between static and dynamic device number allocation.

- *Static*: You pick a fixed major/minor and call register_chrdev_region(MKDEV(major, minor), count, name) — useful when a consistent major is needed.
- *Dynamic*: Kernel assigns a free major using alloc_chrdev_region(&dev, 0, count, name) — avoids conflicts and is recommended for general drivers.

12. Explain the role of insmod and rmmod.

insmod inserts a kernel module into the running kernel (loads .ko and runs its init). rmmod removes an inserted module (calls exit). modprobe is higher-level (resolves dependencies). insmod requires exact path; rmmod removes by module name.

13. What happens inside init_module() and cleanup_module()?

init_module() is the old syscall that backs module initialization — it ultimately calls the module's module_init function. cleanup_module() backs module removal and calls the module's module_exit function. In practice you write your init and exit functions and register them with macros; kernel wraps them.

14. What is the use of MODULE_LICENSE, MODULE_AUTHOR, etc.?

These macros provide metadata about the module: license (GPL vs proprietary — affects exported symbols), author, description, alias. MODULE_LICENSE("GPL") allows use of GPL-only exported symbols; other licenses may restrict. Useful for diagnostics and tools like modinfo.

15. Explain how read(), write(), open(), release() work in a driver.

- open: called when device node is opened (allocate per-file state with file->private_data).

- read: copy data from kernel buffers to user: use copy_to_user. Return bytes read or 0 for EOF. Must handle blocking/non-blocking and waits.
- write: copy data from user to kernel (copy_from_user) and process it. Return # bytes consumed.
- release (close): cleanup file->private_data, stop hardware access if needed. Proper locking and reference counting needed.

16. How do you implement ioctl in a device driver?

Implement .unlocked_ioctl (or .compat_ioctl for 32-bit compat). Define ioctl command numbers using _IO, _IOR, _IOW, _IOWR macros (include type and number). In driver, switch on command number, use copy_to_user/copy_from_user for data transfer. Protect with locking. Example:

```
long my_ioctl(struct file *f, unsigned int cmd, unsigned long arg) {
 switch(cmd) {
  case MY_CMD:
   if (copy_to_user((void __user *)arg, &kernel_struct, sizeof(...))) return -EFAULT;
   break;
 }
 return 0;
}
```

17. How do you use mmap() in a driver?

Expose physical device memory to user-space by implementing .mmap in file_operations. Use remap_pfn_range() (or vm_iomap_memory/io_remap_pfn_range) inside .mmap to map physical pages to the user VMA. Manage vm_operations_struct for open/close if needed. Ensure proper caching attributes (e.g., noncacheable for MMIO).

18. What are atomic operations in kernel space?

Atomic operations (via atomic_t, atomic_read/atomic_set/atomic_inc/atomic_dec, cmpxchg) provide lockless integer ops guaranteed to be atomic across CPUs — used for counters/flags where a full lock is overkill. For 64-bit on 32-bit arch use atomic64_t. Also memory barriers (smp_mb()) and uatomic/arch CMPXCHG exist for complex cases.

SECTION 3 — INTERRUPTS & TIMERS

19. How are interrupts handled in Linux?

When hardware raises IRQ, CPU vector goes to kernel entry; kernel runs the registered top-half interrupt handler (fast, should be minimal). Top-half returns IRQ_HANDLED/IRQ_NONE or defers heavy work to bottom half (tasklet, workqueue, thread). For threaded IRQs, kernel runs handler in a kthread. Drivers register handlers via request_irq() with flags (IRQF_SHARED etc.) and free via free_irq().

20. What is the difference between softirq, tasklet, and workqueue?

- *softirq*: low-level kernel mechanism for bottom halves; runs in softirq context, cannot sleep. Used for networking, timers.
- *tasklet*: built on softirqs — easier API but deprecated directionally; cannot sleep. Tasklets are per-CPU.
- *workqueue*: schedules work in a kernel thread context, so the handler may sleep — safer for longer operations and can use blocking APIs. Use INIT_WORK/queue_work.

21. How to register and free an IRQ in Linux?

Register with request_irq(unsigned int irq, irq_handler_t handler, unsigned long flags, const char *name, void *dev_id); pass dev_id for shared IRQs. Free with free_irq(irq, dev_id). Ensure handler signature and flags (IRQF_SHARED, IRQF_TRIGGER_*).

22. Explain request_irq() and free_irq() functions.

request_irq associates an IRQ number with your handler and reserves it. It can fail (returns negative). free_irq removes the association; it will block/wait until any running handlers complete. Always pair them and check return codes.

23. What is top half and bottom half?

Top half = IRQ handler that runs in interrupt context — must be short and non-blocking. *Bottom half* = deferred work executed later (softirq, tasklet, workqueue) to finish processing in process/thread context or lower-priority context.

24. How to handle shared interrupts?

When IRQs are shared, a driver registers with IRQF_SHARED and supplies a unique dev_id pointer. Handler must check if interrupt belongs to the device (read device status registers)

and return IRQ_NONE if not for it, IRQ_HANDLED if yes. On free_irq provide same dev_id.

25. What are kernel timers and how do you use them?

Kernel timers (struct timer_list) schedule a function to run in softirq context after delay (mod_timer, add_timer). For higher-resolution timers use hrtimer API. To cancel, use del_timer_sync to safely remove and wait for running handler to finish. Timers are useful for periodic work in kernel.

SECTION 4 — KERNEL PROGRAMMING

26. What is a kernel module?

A loadable kernel module (LKM) is binary code (.ko) that can be inserted into a running kernel to extend functionality (drivers, filesystems). It runs in kernel space, has full privileges, and uses module_init/module_exit. Modules can export symbols for other modules using EXPORT_SYMBOL.

27. Explain how printk() works in kernel space.

printk() logs messages to the kernel log buffer; it has priority levels (KERN_EMERG, KERN_ERR, KERN_INFO...). Messages show via dmesg and are routed by syslog to files (e.g., /var/log/kern.log). printk is safe in early boot and interrupt contexts (but formatting is limited). Excessive printk can disrupt real-time behavior.

28. What is kthread and how do you use it?

Kernel threads (kthread) are threads running in kernel context (like user threads but within kernel). Create with kthread_create(threadfn, data, "name") and start with wake_up_process(). Use kthread_should_stop() inside thread loop and stop with kthread_stop(). They can sleep and block safely.

29. How to allocate memory in kernel space?

- kmalloc(size, gfp_flags): contiguous kernel virtual memory suitable for small allocations.
- vmalloc(size): virtually contiguous but physically non-contiguous — for large allocations.

- get_free_pages / __get_free_pages: allocate whole page(s) (power-of-two pages). Choose appropriate GFP_ flags (GFP_KERNEL blocks, GFP_ATOMIC cannot block).

30. Difference between kmalloc(), vmalloc(), and get_free_pages().

- kmalloc: physically contiguous, fast, caches (slab). Best for small objects.
- vmalloc: allocates virtually contiguous memory mapped from multiple pages; used when physical contiguity is not needed and allocation is large. Slower and higher overhead.
- get_free_pages: allocate contiguous page(s) aligned power-of-two; useful for DMA with properly mapped memory or when exact page granularity required.

31. How do you debug kernel code?

Tools/techniques: printk/dmesg, kgdb (remote GDB debugging over serial), kprobe/tracepoints/ftrace for tracing, perf for profiling, systemtap for dynamic tracing, crash/kdump for postmortem, hardware JTAG for core-level debug. Use kernel debug options and dynamic debug (dyndbg), and check /proc/kallsyms & stack traces.

32. What are slab caches?

Slab allocator caches frequently used object sizes for efficiency (kmem_cache_create etc.). Each cache holds pre-initialized objects and reduces allocation overhead and fragmentation. SLAB/SLUB/SLQB are allocator implementations.

33. What is the purpose of spinlock, mutex, and semaphore?

- *Spinlock*: busy-wait lock for short critical sections and usable in interrupt context if not sleeping.
- *Mutex*: sleeping lock for longer operations; cannot be used in interrupt context.
- *Semaphore*: for counting resources, can be used for synchronization that requires more complex semantics (binary semaphore acts like mutex). Use correct primitive depending on whether blocking is allowed and context (interrupt vs process).

34. Can you sleep inside interrupt context?

No — interrupt context cannot sleep. If you need to perform sleeping/blocking work, defer to a workqueue or threaded IRQ where sleeping is allowed.

SECTION 5 — EMBEDDED LINUX & BEAGLEBONE BLACK

35. Explain the booting process of BeagleBone Black.

BeagleBone (AM335x) typical boot: BootROM in SoC reads boot pins and chooses boot device (SD, eMMC, UART). It loads the SPL (MLO) into internal SRAM which initializes DDR and PLLs. SPL loads full U-Boot (u-boot.img) into DRAM. U-Boot initializes peripherals, reads environment (bootcmd), and loads the kernel (zImage/uImage), the DTB (am335x-boneblack.dtb) and optionally initramfs, then runs bootm/bootz. Kernel boots, parses DTB, mounts rootfs (eMMC/SD), and runs init (systemd or BusyBox). (This matches the flow in your question bank.)

36. How is the bootloader different from the kernel?

- *Bootloader*: Small program that runs before kernel; its job is hardware initialization (enough to load kernel), load kernel/DTB/initramfs into memory, and pass control. It may provide interactive prompt, flashing, recovery, network boot.
- *Kernel*: The OS core — initializes full device drivers, virtual memory, scheduling, mounts rootfs and starts user-space. Bootloader is temporary; kernel is persistent.

37. What is the role of MLO and u-boot on BBB?

- *MLO* is the SPL for TI Sitara boards; it fits in the small BootROM-loadable region and initializes DDR.
- *u-boot* (main) is loaded by MLO; it provides a richer environment, devices initialization, and loads kernel/DTB. MLO -> u-boot chain is essential on BBB.

38. What is the purpose of the Device Tree?

Device Tree describes hardware for an architecture-agnostic kernel: CPUs, memory ranges, peripherals (IRQ, reg base), clocks, pinmux, and compatible strings. It allows the same kernel binary to work across boards by providing board-specific hardware description via .dtb.

39. How do you add a custom peripheral to the device tree?

1. Create a node under appropriate bus (e.g., &i2c2 for an I2C device) with compatible = "vendor,device" and properties (reg/address, interrupts, clock-names, gpios).
2. Add driver compatible string to kernel driver of_device_id.

3. Recompile DTB (dtc) or use overlays (.dtbo) and load via capemgr or u-boot. Example snippet:

```
&i2c2 {
  status = "okay";
  mysensor@1a {
    compatible = "acme,mysensor";
    reg = <0x1a>;
    interrupt-parent = <&gpio1>;
    interrupts = <17 IRQ_TYPE_LEVEL_LOW>;
  };
};
```

4. Ensure driver probe registers and binds.

40. What is the use of /dev/mem?

/dev/mem exposes physical memory to user-space (raw mapping) allowing userspace programs to map physical addresses (MMIO). It's powerful but dangerous — security risk and bypasses kernel protection. Prefer kernel drivers or uio/mmap via device-specific drivers instead of /dev/mem.

41. How do you access GPIOs in user space vs kernel space?

- *User-space*: historically sysfs (/sys/class/gpio export/unexport/value/direction), but sysfs-gpio is deprecated in favor of libgpiod (character device /dev/gpiochipN) and gpiod tools. Use device tree overlays to define pinmux/cape.
- *Kernel-space*: implement a gpio_desc or gpio consumer in driver using gpiod interface (gpiod_get, gpiod_direction_output, etc.) or use legacy gpio APIs.

42. What is Cape Manager in BeagleBone?

Cape Manager (capemgr) is a BBB facility to dynamically load Device Tree Overlays (capes) to enable additional hardware (expansion capes). It manages overlay application and resource conflicts. Note: newer kernels and device-tree models shifted to overlay-less or configfs-based approaches; capemgr was common in older BBB images.

43. How do you build and load kernel modules for BBB?

- Cross-compile module against the exact kernel headers/compile tree used for BBB: set ARCH=arm and CROSS_COMPILE=arm-linux-gnueabihf- and build module (make ARCH=arm CROSS_COMPILE=... M=$(pwd) modules).
- Transfer .ko to board and insmod/modprobe. For production add to /lib/modules/$(uname -r)/extra and depmod -a.

44. How to enable or disable I2C/SPI from device tree?

In DTB, set the bus node's status = "okay"; to enable or status = "disabled"; to disable. For dynamic enabling, use overlays that set status or pinmux. For example:

```
&i2c1 {
  status = "disabled";
};
```

Setting to okay and providing pinmux ensures kernel will instantiate those controllers.

SECTION 6 — I2C, SPI, UART DRIVER DEVELOPMENT

45. Explain the I2C protocol.

I²C is a 2-wire serial bus (SDA data, SCL clock) for short-distance communication. Master initiates START, sends 7- or 10-bit address + R/W bit, slave ACKs, then data bytes transferred LSB/MSB with ACKs. Supports multi-master arbitration, clock stretching by slaves. Typical speeds: 100kHz (standard), 400kHz (fast), 1MHz (fast-mode plus), higher modes exist.

46. How does Linux represent I2C devices in the kernel?

Linux represents buses as i2c_adapter (controller) and devices as i2c_client for I2C peripherals. Drivers register as i2c_driver and match devices via device tree compatible or board file entries. The kernel uses i2c-core and higher-level drivers use i2c_smbus_* or i2c_transfer() to exchange messages.

47. What is the i2c_client and i2c_adapter?

- i2c_adapter: kernel object representing the bus controller (hardware, e.g., I2C controller on SoC).

- i2c_client: represents an I2C slave/peripheral device attached to an adapter (has address, device_id, struct i2c_client * passed to driver probe).

48. How do you write a simple I2C driver in Linux?

Skeleton: implement i2c_driver with .probe and .remove, provide of_match_table for DT, register via i2c_add_driver, in probe use i2c_smbus_read_byte_data or i2c_master_recv to read/write. Save pointer to i2c_client in driver private data. Implement sysfs entries or char device to expose functionality.

49. What is the difference between platform_driver and i2c_driver?

They are just different bus-driver bindings: platform_driver binds to platform devices (non-discoverable devices often enumerated by board code or DT). i2c_driver binds via the I2C bus and device addresses. Use the appropriate bus-specific registration so the kernel can match and probe drivers on device discovery.

50. How do you write a UART driver?

Serial drivers integrate with the kernel serial_core and implement uart_driver and uart_ops. Steps: register uart_driver, implement struct uart_ops callbacks (tx_empty, set_mctrl, start_tx, stop_tx, startup, shutdown, set_termios) and register uart_port entries with uart_add_one_port. Or implement a TTY driver directly using the tty_driver/tty_port APIs for low-level serial devices.

51. How is SPI communication different from I2C?

SPI is full-duplex, synchronous serial using at least 4 signals: SCLK, MOSI, MISO, CS (chip select). No addresses — chip selects select specific slaves. SPI is faster and simpler for full-duplex data streams; lacks ACK/NACK and standardized addressing like I2C. I2C allows multi-master and built-in addressing and ACKs; SPI is point-to-point per-CS.

52. Explain spi_sync() and spi_async().

- spi_sync(spi_device, spi_message) executes the transfer synchronously — caller blocks until transfer completes.
- spi_async() queues transfer and returns immediately; completion is notified by a callback in spi_message. Use async for non-blocking transfers.

SECTION 7 — FILE SYSTEM & PROCFS/SYSFS

53. Difference between /proc and /sys?

- /proc: pseudo-filesystem exposing process and kernel status (process info, kernel parameters like /proc/cpuinfo, /proc/<pid>). Primarily for human-readable kernel/proc info.
- /sys (sysfs): structured export of kernel device model (kobjects, attributes) — reflects device-driver hierarchy, allows sysfs attributes to configure and introspect kernel objects (power management, driver binding). Use sysfs for driver attributes, /proc for process/kernel stats.

54. How do you create a proc entry in a driver?

Use proc_create() (modern) with show functions or use seq_file helpers: implement a seq_file show and open via single_open/seq_open, then proc_create("myproc", 0, NULL, &my_fops). On remove, remove_proc_entry("myproc", NULL).

55. What is seq_file API used for?

seq_file simplifies producing sequential textual output for procfs entries that may be large or require iterative reads. It provides start/next/stop/show callbacks so that the kernel can handle large outputs across multiple reads safely and with proper locking.

56. How to expose driver parameters via sysfs?

Use DEVICE_ATTR(name, mode, show, store), then device_create_file(dev, &dev_attr_name). The show callback fills the buffer with current value, store parses user input. On module exit call device_remove_file.

57. How do you create and remove sysfs attributes?

Create using sysfs_create_file(&kobj->kobj, &attr->attr) for kobjects or device_create_file for device-scoped attributes. Remove with sysfs_remove_file or device_remove_file. For multiple attributes, use sysfs_create_group()/sysfs_remove_group().

SECTION 8 — YOCTO / BUILD SYSTEM / CROSS-COMPILATION

58. What is Yocto Project?

Yocto is a build system/meta-build framework to create custom Linux distributions for embedded devices. It provides BitBake, recipes (.bb), layers (meta-*), SDK generation, and reproducible builds tailored to target hardware.

59. What is a recipe in Yocto?

A recipe .bb describes how to fetch, configure, compile, and install a software component (package). A recipe includes variables for SRC_URI, DEPENDS, do_compile/do_install tasks and packaging directives.

60. Explain layers and bitbake.

- *Layers*: Collections of recipes and configuration (meta-). They separate BSPs, distro customizations, and application layers.
- *BitBake*: The build engine that reads recipes and executes tasks (fetch, unpack, configure, compile, package). Use bitbake <image> to build.

61. How do you cross-compile a Linux kernel for ARM?

Set up cross-toolchain: install arm-linux-gnueabihf- cross compiler. Configure kernel: make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- <defconfig>, then make zImage dtbs modules with those env vars. Deploy zImage and DTBs to bootloader-accessible partition.

62. How do you configure a new kernel module in Yocto?

Create a recipe (usually module.bb) for the module or add it into kernel config via kernel fragment. Use IMAGE_INSTALL += "my-module" to include in the image, or create an out-of-tree module recipe that builds against kernel headers provided by Yocto.

SECTION 9 — ARCHITECTURE: ARM & x86

63. What are the key differences between ARM and x86?

ARM is RISC (reduced instruction set), fixed-length instructions, power-efficient, widely used in embedded/mobile. x86 is CISC with variable-length instructions, legacy compatibility and complex addressing modes, common in desktops/servers. Differences also include privilege levels, MMU structures, endianness options, and typical peripheral integration.

64. Explain ARM exception modes.

ARMv7-A supports modes: User, FIQ (fast interrupt), IRQ (interrupt), Supervisor (SVC), Abort (prefetch/data abort), Undefined (undefined instruction), System (privileged user mode), and sometimes Monitor/Hyp for virtualization. Each mode has dedicated banked registers (e.g., FIQ gets banked R8-R12), and exceptions vector to fixed addresses that the CPU jumps to.

65. What is MMU and how does it work in ARM?

MMU (Memory Management Unit) translates virtual addresses to physical addresses via page tables. ARM uses multi-level page tables (e.g., section/descriptors or translation table levels), with domain/access permissions and TLB caching. Kernel sets up page tables and configures TTBR registers; on context switch the kernel changes page tables (or ASIDs in modern ARM) and flushes TLB as needed.

66. What is the use of registers R0 to R15 in ARM?

ARM 32-bit general-purpose registers: R0–R3 pass arguments/return values. R4–R11 generally callee-saved. R12 is IP (intra-procedure call scratch). R13 = SP (stack pointer), R14 = LR (link register), R15 = PC (program counter). Usage conventions vary by ABI but these are common.

67. What happens when an interrupt occurs in ARM?

CPU switches to appropriate exception mode (IRQ/FIQ), saves CPSR to SPSR of that mode, sets LR to return address, disables further interrupts of that type (unless configured), and jumps to exception vector. Handler executes, acknowledges the interrupt in controller, and returns by restoring CPSR/PC.

68. What are the different types of instructions in x86?

x86 instruction types include data movement (MOV), arithmetic (ADD, SUB, IMUL), logical (AND, OR, XOR), control flow (JMP, CALL, RET), string operations, I/O instructions (IN/OUT), and system instructions (INT, SYSENTER, SYSCALL). x86 also has SIMD instructions (SSE/AVX).

69. How does stack management differ in ARM and x86?

Both use a stack and SP register (R13 for ARM, RSP/ESP for x86). Differences: ARM calling conventions and register usage differ, ARM often uses LR for return address rather than pushing PC, and typical ARM code uses push/pop multiple registers via stmdb/ldmia. x86 pushes return address via call onto stack; ARM's bl stores return in LR. Stack alignment rules and calling conventions differ per ABI.

70. What is a pipeline and how does it work in modern CPUs?

A pipeline breaks instruction execution into stages (fetch, decode, execute, memory, writeback) so multiple instructions overlap in flight. Modern CPUs have deep pipelines, superscalar execution (multiple instructions per cycle), branch prediction, speculative

execution, and out-of-order execution to increase throughput. Hazards (data, control) and caches/TLBs affect performance.

SECTION 10 — DEBUGGING, PERFORMANCE & TOOLS

71. How to debug kernel oops?

Read oops/panic log from serial or dmesg. Use the backtrace addresses and kernel symbol table (/proc/kallsyms) or ksymoops/addr2line to map addresses to source file/line if you have vmlinux with symbols. Identify problematic driver/module and reproduce under instrumentation (dynamic debug, ftrace). Use echo 1 > /proc/sys/kernel/printk and enable CONFIG_DEBUG_INFO for more helpful traces.

72. What is kgdb and how do you use it?

KGDB allows debugging a live kernel with GDB over serial/Ethernet. Enable CONFIG_KGDB, boot kernel with kgdboc=ttyS0,115200 and use gdb vmlinux on host connecting to target serial. You can set breakpoints in kernel code, examine memory and registers, and single-step.

73. What are common causes of kernel panics?

NULL pointer dereferences, use-after-free, stack overflow, bad pointer access, corrupted data structures, double-free, mismatched memory mapping for DMA, module ABI mismatch, and hardware faults. Drivers running in wrong context (sleeping in IRQ) or bad device tree mismatches can also cause panics.

74. How to detect memory leaks in kernel?

Use kmemleak (enable CONFIG_DEBUG_KMEMLEAK) which reports possible leaks, use slabtop to monitor kernel object allocations, vmstat/free for system memory patterns, or dynamic debug and trace allocations. For modules, ensure paired kmalloc/kfree and check dmesg for [kmemleak] messages.

75. How does strace, perf, or ftrace help in performance profiling?

- strace: traces user-space syscalls, useful to see blocking syscalls and syscall frequency.
- perf: powerful profiler for CPU cycles, stack traces, hardware events, and software events; use perf record/perf report.

- ftrace: kernel tracer for functions/events and low-overhead instrumentation (trace-cmd, kernelshark), good for kernel function timing and hotspot analysis.
